

CS 5200 HW 3

Each problem carries 10 points. Total 40 points. Two students can make a team and submit one assignment.

Problem 1: 3-SUM Problem: Use Hashing. Note: If you don't use hashing in your program, then it is not an acceptable answer and there will be no points.

Part A) Given a list of integers (randomly arranged), check if the array contains 3 numbers x , y and z such that $x+y+z = \text{target}$.

This is very similar to 2 sum, but with basically an extra step. You can just use modified subtreesic:

1. Use a for loop to iterate through the first number x in $\text{arr}[i]$
2. For each x , the problem reduces to finding two other numbers y and z in the remainder of the array such that $y + z = \text{target} - x$.
3. This is literally just the 2-SUM problem on the remainder of the array
 - a. Create an empty HashSet
 - b. Iterate through the remaining elements in $\text{arr}[j]$
 - c. For each element, calculate the $\text{neededValue} = (\text{target} - x) - \text{arr}[j]$
 - d. If neededValue is already in the HashSet, found 3 numbers. Return true.
 - e. If not, add $\text{arr}[j]$ to the HashSet and continue

Part B) What is the time complexity of your method?

Since 2-SUM algorithm takes $O(n)$ and it is run n times (once for each x). 3-SUM takes $O(n^2)$ time using hashing.

Boolean or Integer checkTripleSum(int []arr, int target) {}

```
checkTripleSum(int[] arr, int target) {  
    int n = arr.length;
```

```

// only go up to n-2 because need at least 3 elements
for (int i = 0; i < n - 2; i++) {

    int currentTarget = target - arr[i];

    // use a HashSet to solve 2-SUM for the remaining elements
    HashSet<Integer> set = new HashSet<>();

    for (int j = i + 1; j < n; j++) {
        int neededValue = currentTarget - arr[j];

        // if the needed value is in the set, found valid 3sum
        if (set.contains(neededValue)) {
            // System.out.println("Triplet found: " + arr[i] +
            ", " + arr[j] + ", " + neededValue);
            return true;
        }

        // otherwise add the current element to the set so it
        can be matched later
        set.add(arr[j]);
    }

    // if the loop finishes without returning true, no triplet
    exists
    return false;
}

```

Example 1:

```

boolean answer = checkTripleSum( arr, 29 );

```

answer will be false here because no combination of 3 numbers in the array has sum of 29

Example 2:

```

boolean answer = checkTripleSum( arr, 9 );

```

answer will be true in this case because $(3+2+4) = 9$

Problem 2:

There are two friends John and Jill who go to a restaurant and order their food. John orders six food items worth \$4, \$1, \$2, \$1, \$1 and \$2. John has to pay \$11 in total. Jill orders four Italian food items worth \$3, \$6, \$3, and \$3. Jill has to pay \$15. As they are eating the delicious food, a question comes to Jill's mind. If John and Jill exchange an item from their order, they would have to pay the same amount. For instance, if John gives an item of \$1 to Jill and Jill reciprocates by giving John an item of \$3, both of them would have to pay \$13 each.

John's new total = $\$11 - \$1 + \$3 = \13

Jill's new total = $\$15 - \$3 + \$1 = \13

Part A) Write a hashing based program that solves this riddle for John and Jill. The output of the program is a pair of item that needs to be exchanged between John and Jill to make their bill the same amount.

Part B) What is the time complexity?

The time complexity is influenced by 3 main parts. Firstly, the $O(n + m)$ for linearly scanning in John's and Jill's orders. Secondly, the $O(m)$ to build Jill's hashset. And finally, the $O(n)$ loop over John's n items when checking to see if Jill has them ($O(1)$ lookup). The time complexity is dominated by the $O(n + m)$

```
void printPairToExchange( int []JohnBill, int []JillBill )
{
    // calculate both sums
    int sumJohn = 0;
    for (int item : JohnBill) sumJohn += item;

    int sumJill = 0;
    for (int item : JillBill) sumJill += item;
```

```

        // check if a clean swap is even mathematically possible
        if ((sumJohn - sumJill) % 2 != 0) {
            System.out.println("It is impossible to balance the bills
with these items.");
            return;
        }

        // calculate the target difference
        int targetDiff = (sumJohn - sumJill) / 2;

        // put all of Jill's items into a HashSet for O(1) lookups
        HashSet<Integer> jillSet = new HashSet<>();
        for (int b : JillBill) {
            jillSet.add(b);
        }

        // loop through John's items and look for the perfect match
        for (int a : JohnBill) {
            int neededB = a - targetDiff;

            // if Jill has the item we need
            if (jillSet.contains(neededB)) {
                System.out.println("John should give $" + a + " and
Jill should give $" + neededB);
                return;
            }
        }

        System.out.println("No matching pair found.");
    }

    main()
    {
        int[] JohnBill = {4, 1, 2, 1, 1, 2}; // sum = 11
        int[] JillBill = {3, 6, 3, 3};        // sum = 15

        printPairToExchange(JohnBill, JillBill);
    }

```

Problem 3. Implement a Bloom Filter. For any example of your choice, compare your Bloom Filter implementation to a traditional hash table. You can use a library function (e.g., `unordered_map` in C++ or in python) for hash table. Mention the space and time complexity aspects of bloom filter compared to hash table for your use case. You can use the internet for help regarding Bloom Filter implementation. Use either C/C++ or Python for implementation. The purpose of this assignment is get experience/familiarity with bloom filter implementations.

```
import hashlib
import sys

class BloomFilter:
    def __init__(self, size, num_hash_functions):
        self.size = size
        self.num_hash_functions = num_hash_functions
        # initialize a bit array of the given size, all set to 0
        self.bit_array = [False] * size

    def _hash(self, item, i):
        # create k different hash functions by appending 'i' to the
item
        # use MD5, convert it to an integer, and modulo by the array
size
        hash_string = str(i) + item
        result = hashlib.md5(hash_string.encode())
        return int(result.hexdigest(), 16) % self.size

    def add(self, item):
        # hash the item k times and set those bits to 1
        for i in range(self.num_hash_functions):
            index = self._hash(item, i)
            self.bit_array[index] = True
```

```

def check(self, item):
    # hash the item k times. If any bit is False, it is not in the
set
    for i in range(self.num_hash_functions):
        index = self._hash(item, i)
        if not self.bit_array[index]:
            return False
    # If all bits are True, it is possibly/prob in the set
    return True

if __name__ == "__main__":
    malicious_urls = [
        "http://evil.com", "http://scam.net", "http://phishing.org",
        "http://hack.io", "http://malware.com",
"http://stealyourdata.biz"
    ]

    # It is a little differnt than c++ because Hash Tables in python
are just called sets
    hash_table = set()
    for url in malicious_urls:
        hash_table.add(url)

    # a Bloom Filter with a Size of 50 bits and using 3 Hash
Functions
    bloom_filter = BloomFilter(size=50, num_hash_functions=3)
    for url in malicious_urls:
        bloom_filter.add(url)

    test_url_1 = "http://evil.com"          # in the dataset
    test_url_2 = "http://google.com"        # not in the dataset

    print(f"Testing known malicious URL: {test_url_1}")
    print(f"Hash Table says: {test_url_1 in hash_table}")
    print(f"Bloom Filter says: {bloom_filter.check(test_url_1)}\n")

    print(f"Testing clean URL: {test_url_2}")

```

```

print(f"Hash Table says: {test_url_2 in hash_table}")
print(f"Bloom Filter says: {bloom_filter.check(test_url_2)}")

print("\n=== Memory Usage ===")
print(f"Hash Table Size: {sys.getsizeof(hash_table)} bytes")
print(f"Bloom Filter Bit Array Size:
{sys.getsizeof(bloom_filter.bit_array)} bytes")

```

Complexity Comparison

Time Complexity:

- Hash table: $O(1)$ average time for both adding to the hash table and checking whether it contains an element or not. However, in the worst case scenario where the hash table is mostly full and has a lot of collisions, the time complexity can degrade to $O(N)$.
- Bloom filter: $O(k)$ time for both adding to the bloom filter and checking whether it contains an element or not. However, if/because k is small and a fixed constant it effectively becomes $O(1)$ time. Furthermore, a bloom filter never degrades to $O(n)$ lookup time since its performance is strictly independent of the number of items in it.

Space complexity:

- Hash table: $O(N \times L)$ where N is the number of items and L is the size of the strings. A hash table must store the actual data of the items to be able to resolve collisions. This takes up a lot of memory for a large amount of data.
- Bloom filter: $O(m)$ bits where m is the size of the bit array. What is nice is the bloom filter's space complexity is completely independent of the size of the input (Length of the string).

Accuracy Trade-off:

- Hash table: hash tables are 100% accurate
- Bloom filters: Are 100% accurate for true negatives but it does allow for false positives.

This is a good tradeoff because you get massive gains in space efficiency.

Problem 4

In class we discussed how cryptographic hashing can be used to securely transfer a large file by allowing us to verify if the file has been tampered (modified). Hash of the file can be generated by separately hashing the file chunks and assembling the hash values in a binary tree fashion to generate a top hash (root hash). Write a program to create a binary tree of hash values of individual file chunks. This concept is called Hash tree. See Wikipedia article on Merkle tree for reference.

The leaves of this tree correspond to the hash values of the file chunks. Files have to be generated for Hashing. Hash function is used by invoking the "digest" method as shown in a sample program:

CryptoHashing.java

Then for every pair of leaves, create a parent node:

```
parent.hashValue = hash_function( child1.hashValue + " " +
child2.hashValue );
```

This happens recursively for all internal nodes of the tree, including the root node.

createHashTree method below returns the root node of a hash tree built using hashing of input filenames as output. Each internal node of the tree contains the hash value as a String data type of its two children

Implement this method which just returns the root's hash value (which is a string)

```
String createHashTree(List<String> filenames) { }
```

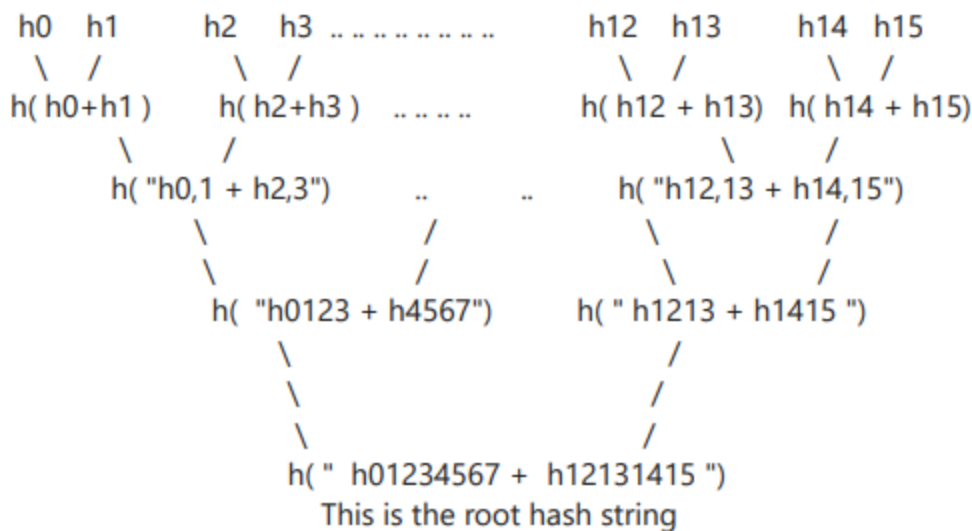
The files are hashed only once in the beginning as I am showing here

```
Input file[0] -> Output is hash of file[0] => String h0
Input file[1] -> Output is hash of file[1] => String h1
Input file[2] -> Output is hash of file[2] => String h2
Input file[3] -> Output is hash of file[3] => String h3
Input file[4] -> Output is hash of file[4] => String h4
Input file[5] -> Output is hash of file[5] => String h5
Input file[6] -> Output is hash of file[6] => String h6
Input file[7] -> Output is hash of file[7] => String h7
Input file[8] -> Output is hash of file[8] => String h8
Input file[9] -> Output is hash of file[9] => String h9
Input file[10] -> Output is hash of file[10] => String h10
Input file[11] -> Output is hash of file[11] => String h11
Input file[12] -> Output is hash of file[12] => String h12
Input file[13] -> Output is hash of file[13] => String h13
Input file[14] -> Output is hash of file[14] => String h14
Input file[15] -> Output is hash of file[15] => String h15
```

Above steps are what a base case of a recursive algorithm will do. In a tree, these are the leaf level computations

As you can see, the output are 16 String values; namely h0, h1, ..., h14, h15. After that hashing happens on these 16 hash values in a pair-wise fashion; similar to a tournament tree

In the diagram below, h() means a hash function. In my sample code, hash function is hashStringValue(.) method. Hash function is called on concatenated strings. Some nodes are omitted for brevity.



$h(h_0+h_1)$ means hash function on concatenation of h_0 string and h_1 string. If $h_0 = \text{"abcd"}$ and $h_1 = \text{"xyzw"}$, then the parent of the two children h_0 and h_1 will be $h_{0,1} = \text{hash_function}(\text{"abcd" + "" + xyzw})$

Similarly, other parent nodes like $h_{2,3}$ are computed. As you can see, the files are not touched in the internal nodes. For internal nodes, hashing only happens on hash value strings.

My implementation:

```
import java.util.List;
import java.util.Arrays;
import java.security.MessageDigest;

class MerkelTree {

    public String createHashTree(List<String> filenames) {
        if (filenames == null || filenames.isEmpty()) {
            return "";
        }
        return buildHashTree(filenames, 0, filenames.size() - 1);
    }

    public String buildHashTree(List<String> files, int low, int high)
    {
```

```

        // leaf node, hash it
        if (low == high) {
            return hashStringValue(files.get(low));
        }

        int mid = (low + high) / 2;

        String left  = buildHashTree(files, low,      mid);
        String right = buildHashTree(files, mid + 1,  high);

        // concat the hashes
        return hashStringValue(left + right);
    }

    // this stuff in from CryptoHashing.java. I jst modified it for my
    use
    public String hashStringValue(String input) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-1");
            byte[] hashBytes =
digest.digest(input.getBytes("ISO-8859-1"));

            StringBuilder hexString = new StringBuilder();
            String hex = "0123456789abcdef";

            for (int i = 0; i < hashBytes.length; i++) {
                if (i % 4 == 0) {
                    hexString.append(" ");
                }
                hexString.append(hex.charAt((hashBytes[i] & 0xF0) >>
4)); // high nibble
                hexString.append(hex.charAt((hashBytes[i] & 0x0F)));
// low nibble
            }
            return hexString.toString().trim();
        } catch (Exception e) {
            throw new RuntimeException("Hashing failed: " +
e.getMessage());

```

```

    }
}

class Main {
    public static void main(String[] args) {
        MerkelTree tree = new MerkelTree();

        // i'm not really sure how you want us to show it works so
        here are 16 files/strings
        List<String> filenames = Arrays.asList(
            "file0.txt",
            "file1.txt",
            "file2.txt",
            "file3.txt",
            "file4.txt",
            "file5.txt",
            "file6.txt",
            "file7.txt",
            "file8.txt",
            "file9.txt",
            "file10.txt",
            "file11.txt",
            "file12.txt",
            "file13.txt",
            "file14.txt",
            "file15.txt"
        );

        // create tree which also creates the hashes for all the sub
        trees as they are used to compute the root
        String rootHash = tree.createHashTree(filenames);
        System.out.println("Merkle Root Hash: " + rootHash);
    }
}

```